

C  mputo Concurrente 2026

Pr  ctica 3

Exclusi  n Mutua: Candados Cl  sicos y el Modelo de Memoria de Java (JMM)

Profa: Gilde Valeria Rodr  guez

1 Contexto

Para que un programador pueda dise  ar y razonar de manera concurrente sobre un sistema necesita conocer el *modelo de memoria*. Existen modelos de memoria a nivel de hardware (por ejemplo, el TSO de la m  quina SPARC) y tambi  n a nivel de software (por ejemplo, el de Java, el de C++).

Informalmente, un modelo de memoria especifica como las acciones en memoria (por ejemplo, las lecturas y escrituras) en un programa parecen ejecutarse desde el punto de vista del programador, por ejemplo, el valor que debe regresar la lectura de una determinada ubicaci  n en memoria.

En un programa de un solo hilo, el modelo de memoria es trivial, ya que se debe leer lo   ltimo escrito. Sin embargo, en un programa multihilo el modelo de memoria es m  s complejo, ya que una lectura de un hilo puede ver o no escrituras de otros [MLS24].

Para un lenguaje de programaci  n de alto nivel como Java, el modelo de memoria determina las transformaciones que el compilador puede aplicar a un programa al producir el bytecode (por ejemplo: <https://gee.cs.oswego.edu/dl/jmm/cookbook.html>), las transformaciones que la m  quina virtual puede aplicar al producir el c  digo nativo, y finalmente, las optimizaciones que el hardware puede realizar en el c  digo nativo [MPA05].

Es por esto que como programadores necesitamos conocer el modelo de memoria, ya que las transformaciones que se realizan en el c  digo, pueden impactar el resultado de nuestros programas. Sin un modelo de memoria bien especificado o si no lo conocemos, es imposible conocer los resultados de un programa.

El modelo de memoria de Java se ha estudiado ampliamente desde los 90's [MPA05], de forma breve podemos resaltar dos puntos importantes sobre el modelo que nos afectan como programadores de Java: el Reordenamiento y la Visibilidad.

Reordenamiento. El compilador, con el objetivo de optimizar, puede intercambiar el orden de dos l  neas de c  digo de un programa concurrente, por ejemplo, en el candado Peterson nos representa un problema si el compilador considera cambiar el orden de $flag[i] = true$; y $victim = i$;

Visibilidad. Como cada hilo tiene copias de los objetos compartidos en la Heap, puede ser que la escritura del objeto no se refleje de forma enseguida en la lectura de ese mismo objeto por otro hilo.

Para mitigar este par de problemas se utilizan los campos *volatile* y *final*, e incluso los objetos *at  micos*. Los campos *volatile* y *final* garantizan que la actualizaci  n de una variable se refleje enseguida en todas sus copias, y que se respete el orden de las instrucciones en los programas.

2 Ejemplos

En el siguiente link: https://github.com/surindt/FC_CConcurrente/tree/main/Programas_P3

- Programa 1 (VolatileExample1): Programa que ejemplifica el problema de Visibilidad en el JMM.
- Programa 2 (VolatileExample2): Programa que ejemplifica el problema de Reordenamiento en el JMM.
- Programa 3 (LockPeterson): Programa que ejecuta la clase Peterson, todas las variables utilizan volatile
- Programa 4 (ExecuteBakery): Programa que ejecuta la clase Bakery, implementa Bakery de forma no acotada (sin overflow). Utiliza objetos atómicos.

3 Ejercicios

- Entrega en un pdf las respuestas de los ejercicios que no requieran implementarse y añade una breve descripción de los programas que entregas (sus nombres y qué hacen).
- Los programas deben ir en un zip. Debes realizar un programa en cada ejercicio que indique “Implementar”.
- Recuerda que debes utilizar Java 21 LTS.
- **Si no compila utilizando Java 21LTS o si no se entrega la breve descripción de los programas se penalizará.**

Tiempo de elaboración: $\approx$ 2hr

Total de puntos: 100

Importante: No utilicen en su contador la paquetería Atomic, en general no la utilicen. Cuando utilizan `compareAndSet()`, `get()`, `getAndIncrement()`, etc, están haciendo que todo lo que está adentro de estos métodos se haga de forma atómica. Por ejemplo, si utilizan `getAndIncrement()` en su contador, este se vuelve linealizable, es decir, con consistencia, siempre contará bien, entonces no tendría caso utilizar candados y ese es el objetivo de esta práctica.

También tengan cuidado con otros objetos de la biblioteca Concurrent de java, muchas de estas implementaciones ya son linealizables, y en esta práctica no las necesitamos.

Solo ocupan **volatile**.

1. El algoritmo de candado Peterson solo funciona para 2 hilos, utiliza el algoritmo de candado Peterson para implementar un algoritmo de candado que funcione para 4 hilos. *Hint: Apóyate del programa Peterson*
 - (a) Para probar que tu candado funciona, crea una implementación en donde utilices un `ExecutorService` para ejecutar 400 tareas, cada tarea debe aumentar en uno un objeto Contador. Utiliza tu candado para 4 hilos para tener consistencia en tu Contador.
 - (b) De alguna forma obtén el número de veces que los hilos aumentan el contador. ¿Cada uno realiza exactamente 100 tareas o hay algunos que realizan más?
 - (c) ¿Consideras que la implementación cumple con Justicia? Justifica tu respuesta.

Advertencia: Puedes considerar el pseudocódigo de la Tarea 3 (`DoublePeterson`). Solo ten cuidado con los **id's**, ya que si utilizas modulo 2 para `Peterson` y modulo 4 para `DoublePeterson`, puede pasar que dos hilos ejecuten un candado Peterson con el mismo id. Si dos hilos en Peterson tienen el mismo id entonces no se cumple exclusión mutua, y el candado `DoublePeterson` no cumplirá su función, no contará las 400 tareas.

2. En base a la implementación de Bakery vista en la clase teoría, implementa Bakery para 4 hilos. *Hint: Crea los arreglos `flag` y `label` de tamaño 4, si consideras necesario utiliza campos volatile*
 - (a) Con ayuda de un `ExecutorService` ejecuta 400 tareas, cada tarea debe aumentar en uno un objeto Contador. Utiliza tu candado para 4 hilos para tener consistencia en tu Contador.
 - (b) De alguna forma obtén el número de veces que los hilos aumentan el contador. ¿Cada uno realiza exactamente 100 tareas o hay algunos que realizan más?
 - (c) ¿Consideras que la implementación cumple con Justicia? Justifica tu respuesta.
3. Revisa el programa de Bakery que se les compartió como ejemplo, ejecútalo varias veces, revisa el código y contesta:
 - (a) Revisa para que sirve el campo `AtomicReference`, y qué hacen los métodos `compareAndSet()` y `get()`.
 - (b) Describe como funciona en a lo más 6 líneas de computadora.
 - (c) ¿Si `next` no es un `AtomicReference` sigue funcionando? *Hint: Recuerda la Cola concurrente sin candados que implementaste en la Práctica 2*
 - (d) ¿Consideras que mantiene la lógica de la implementación vista en clase/tu implementación del ejercicio anterior? Justifica porqué.

References

- [MLS24] Trevor Brown Michael L. Scott. *Shared-Memory Synchronization*. Springer Cham, 30 January 2024.
- [MPA05] Jeremy Manson, William Pugh, and Sarita V. Adve. The java memory model. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '05, page 378–391, New York, NY, USA, 2005. Association for Computing Machinery.